

Traversing the Pump/Gain Map: Continuation and Warm-Start Strategies in `twpa_solver` Current Scheme and a Survey of Alternatives

TWPA solver — `twpa_jax`

July 10, 2026

Abstract

The gain map produced by `scripts/run_gain_map.py` requires a converged harmonic-balance (HB) pump solution at every cell of a two-dimensional (pump-power $\times$ pump-frequency) grid. Because each cell is a stiff nonlinear root-find, the map is not solved cell-by-cell from scratch; instead the solver *continues* solutions from one operating point to the next. This report documents exactly which continuation and warm-start strategies the code uses today, locates each in the source, and then surveys more sophisticated alternatives from the numerical-continuation literature (pseudo-arclength, tangent predictors, deflated continuation, pseudo-transient continuation, and 2-D map-traversal schemes), with references. It is descriptive: it maps the current design and catalogues options, and does not prescribe a change.

Contents

1 Problem setup

1.1 The harmonic-balance residual

Each map cell fixes a pump frequency f_p (angular $\omega_p = 2\pi f_p$) and a pump drive amplitude I_p . The pump state is the vector of complex harmonic coefficients $\mathbf{X}$ of the node fluxes on the mode set $\{k\omega_p\}$. The steady state solves a nonlinear algebraic residual

$$\mathbf{R}(\mathbf{X}; \lambda) = 0, \quad \mathbf{R}(\mathbf{X}; \lambda) = \mathcal{L}(\omega_p) \mathbf{X} + \mathcal{N}(\mathbf{X}) - \lambda \mathbf{S}, \quad (1)$$

where $\mathcal{L}(\omega_p)$ is the frequency-domain linear block, $\mathcal{N}$ the Josephson (cosine) nonlinearity evaluated through the alias-free time transform, $\mathbf{S}$ the pump source phasor, and $\lambda \in [0, 1]$ a *scalar homotopy / continuation parameter* that scales the drive. At $\lambda = 1$ the physical problem is recovered; at $\lambda \rightarrow 0$ the problem is linear and trivially solvable. The source enters *linearly* in λ (Eq. ??), which is the property the whole continuation machinery exploits (`src/twpa_solver/pump/solver.py`).

1.2 Two nested traversal problems

There are two distinct “traversal” questions, and the code answers them with different mechanisms:

1. **Intra-cell (parameter homotopy).** How do we get *one* cell from the easy linear regime up to full drive $\lambda = 1$? — answered by *continuation in λ* (§??).
2. **Inter-cell (map traversal).** How do we use a *neighbouring* converged cell to cheapen the next cell? — answered by *warm-starting* plus an extrapolation predictor along the power axis (§??).

2 What we do now

2.1 Intra-cell: natural-parameter continuation in the pump amplitude

The continuation parameter is the pump drive amplitude itself (via λ), i.e. *natural-parameter continuation*. Two modes exist.

Fixed-step continuation (the “cold” reference). `solve_continuation` (`solver.py:402`) walks a fixed ladder $\lambda \in \{1/N, 2/N, \dots, 1\}$ with $N = 20$ steps (`--continuation-steps`, default 20). At each step it runs one Newton–Krylov solve seeded from the previous converged step. If any step fails it returns immediately (no re-convergence is attempted). This is the trusted, deterministic path used to validate everything else and is selected by `pump_flags_cold` (`run_gain_map.py:252`).

Adaptive-step continuation (the “seed” path). `solve_adaptive_continuation` (`solver.py:463`) replaces the fixed ladder with a step controller: start at `--adaptive-initial-step`, on a converged step grow the step by $\times 1.5$, on a failed step shrink by $\times 0.5$; if the step falls below `--adaptive-min-step` it falls back to a fixed 20-step run. This is the first-point path for each frequency column in the warm pass, seeded from the linear phasor guess (below).

The Newton–Krylov corrector. Every continuation step calls `solve_one` (`solver.py:101`): a damped Newton iteration whose linear systems are solved matrix-free by GMRES against the true Jacobian $\mathbf{J} = \partial \mathbf{R} / \partial \mathbf{X}$, preconditioned by one of several factorizations (`--inproc-preconditioner`). Cost-control knobs live here: a stall detector (`stall_ratio`, `stall_patience`) and a wall-clock `solve_deadline_s` abort stiff *over-fold* solves, and modified-Newton preconditioner reuse (`precond_reuse`) amortizes the LU.

Predictors inside continuation. At each λ step the initial guess is, by default, simply the previous converged state (a *trivial / zeroth-order predictor*). Optionally a *secant predictor* extrapolates linearly in λ from the last two converged states (`solver.py:422` and `:501`):

$$\mathbf{X}_{\text{guess}} = \mathbf{X}_{\text{prev}} + \beta (\mathbf{X}_{\text{prev}} - \mathbf{X}_{\text{prevprev}}), \quad \beta = \frac{\lambda - \lambda_{\text{prev}}}{\lambda_{\text{prev}} - \lambda_{\text{prevprev}}}. \quad (2)$$

The linear phasor seed (homotopy start). `build_linear_phasor_seed` (`seeds.py:46`) solves the *first-harmonic linear block only*, $\mathcal{L}_1(\omega_p) \mathbf{X}_1 = \mathbf{S}_1$, by GMRES, giving a physically-motivated non-zero start for the adaptive path. This is the $\lambda \rightarrow 0$ analytic solution used as the continuation origin (as opposed to the all-zeros cold start).

2.2 Inter-cell: warm-starting across the map

The map is *not* solved cell-independently. The production path is the in-process warm pass `run_warm_pass_inprocess` (`run_gain_map.py:849`), which traverses the grid as follows:

1. **Column-major, increasing power.** Cells are grouped by frequency column (`by_col`); each column is sorted by ascending pump power and traversed low $\rightarrow$ high (`run_gain_map.py:871--880`).
2. **First cell of a column: seed.** Solved from the `linear_phasor` seed with adaptive continuation (mode “seed”).
3. **Subsequent cells: warm start.** Each higher-power cell is solved by a *single full-scale Newton solve at $\lambda = 1$* , `solve_direct` (`solver.py:383`), initialised from the previous converged cell’s HB state — *no* intra-cell continuation ladder. This is the main speed-up: one Newton solve instead of ~ 20 .

4. **Secant predictor along the power axis.** Between cells the guess is extrapolated from the last two converged solutions along the pump-*current* axis, `secant_guess` (`run_gain_map.py:785`, default `--inproc-fold-predictor secant`). Structurally identical to Eq. ?? but with the pump current as the abscissa. A bad extrapolation is caught and retried from the plain warm start (:901).
5. **Recovery ladder.** A failed warm solve (non fail-fast) retries as a full "seed" adaptive solve (:906); the subprocess path adds a one-shot reseed (`run_gain_map.py:424`).

Where the traversal stops: the harmonic-balance fold. As pump power rises a column reaches a *turning point* (fold) of Eq. ?? : above it the branch has no nearby solution and Newton cannot re-converge. The code does not attempt to pass this fold. Instead it detects repeated failures and culls the rest of the column: `--fold-skip-patience` (default 2) marks all higher-power cells `SKIP_PAST_FOLD` (gain NaN) without solving (`past_fold_skip_row`, `run_gain_map.py:826`). This is a known sharp edge: at broadband operating points the true solution can need ~ 15 Newton iterations, and a too-tight Newton cap plus `fold-skip-patience 2` can *false-trip* and skip the genuine broadband region before the real fold.¹

One important structural fact. Warm-starting is *purely one-dimensional*: it chains *within* a frequency column along the power axis. Each new frequency column is restarted from the linear phasor seed — there is *no* warm start from the adjacent converged *column*. The 2-D grid is thus traversed as a set of independent 1-D sweeps. This is the single biggest lever the alternatives in §?? act on.

2.3 Code map

Concept	Location	Role
Fixed-step continuation	<code>solver.py:402</code>	cold reference, $N=20$ λ ladder
Adaptive-step continuation	<code>solver.py:463</code>	step grow/shrink + fixed fallback
Single warm Newton solve	<code>solver.py:383</code>	inter-cell warm start at $\lambda=1$
Newton–Krylov corrector	<code>solver.py:101</code>	GMRES on true Jacobian, pre-cond.
Secant predictor (in λ)	<code>solver.py:422, 501</code>	1st-order extrapolation in continuation
Linear phasor seed	<code>seeds.py:46</code>	homotopy start $\mathcal{L}_1 \mathbf{X}_1 = \mathbf{S}_1$
Warm pass (2-D traversal)	<code>run_gain_map.py:849</code>	column-major, low→high power
Secant predictor (in I_p)	<code>run_gain_map.py:785</code>	inter-cell power-axis extrapolation
Fold short-circuit	<code>run_gain_map.py:826</code>	<code>SKIP_PAST_FOLD</code> culling

3 Why the fold is the crux

Natural-parameter continuation parameterizes the solution branch by the physical parameter (here pump amplitude). At a fold the branch is *vertical* in that parameter: $\partial \mathbf{X} / \partial \lambda \rightarrow \infty$ and the Jacobian $\mathbf{J}$ becomes singular. Any method that insists on advancing the parameter monotonically *must* stall there — which is exactly why the current code treats the fold as a hard wall and culls beyond it. The literature’s central message is that this wall is an artefact of the *parameterization*, not of the physics: reparameterizing the branch (by arclength) lets a solver round the fold and continue onto the upper branch [?, ?]. Whether the upper (post-fold) branch is physically wanted for a TWPA gain map is a separate design question; but even if only

¹Documented internally: the false-trip culls the 2c broadband operating point; mitigations are looser Newton caps and higher fold-skip patience.

the lower branch matters, a fold-aware method locates the fold precisely instead of bracketing it with a heuristic patience counter.

4 Alternatives from the literature

4.1 Pseudo-arclength continuation (round the fold)

Instead of fixing λ and solving for $\mathbf{X}$, treat *both* $\mathbf{X}$ and λ as unknowns and add one scalar constraint that advances a *pseudo-arclength* s along the branch [?]:

$$\begin{cases} \mathbf{R}(\mathbf{X}, \lambda) = 0, \\ \dot{\mathbf{X}}_0^\top (\mathbf{X} - \mathbf{X}_0) + \dot{\lambda}_0 (\lambda - \lambda_0) - \Delta s = 0, \end{cases} \quad (3)$$

where $(\dot{\mathbf{X}}_0, \dot{\lambda}_0)$ is the unit tangent at the current point. The augmented $(N+1) \times (N+1)$ system stays non-singular *through* the fold because the branch is never vertical in s . This is the standard way HB frequency-response and force-response curves are traced past turning points in mechanics and RF/microwave engineering [?, ?]. Cost: the augmented system and the tangent solve; benefit: no fold heuristic, and the fold power is found exactly.

4.2 Tangent (Euler) predictor vs. the current secant

The code’s predictors are trivial (copy) and secant (Eq. ??). The next step up is the *tangent / Euler predictor*: solve $\mathbf{J} \dot{\mathbf{X}} = \partial \mathbf{R} / \partial \lambda$ once to get the true branch tangent and predict $\mathbf{X}_{\text{guess}} = \mathbf{X}_0 + \Delta \lambda \dot{\mathbf{X}}_0$ [?]. It is first-order-accurate like the secant but uses derivative information rather than a finite difference of past points, so it is robust when the previous two points are far apart or the branch curves sharply — precisely the near-fold regime. `BifurcationKit.jl`’s predictor page is a compact catalogue of secant/tangent/Bordered/polynomial predictors and their trade-offs [?].

4.3 Affine-invariant adaptive step control

The current adaptive controller uses fixed $\times 1.5 / \times 0.5$ heuristics. Deuffhard’s affine-invariant Newton theory gives a principled step-length and damping controller driven by the observed Newton contraction, with convergence monitors that resize the continuation step from the corrector’s behaviour rather than a fixed ratio [?]. This tends to take large steps on the flat part of a column and automatically shorten them approaching the fold.

4.4 Pseudo-transient continuation (Ψtc)

For a single stiff cell, pseudo-transient continuation embeds the root-find in a fake time march, $\frac{\mathbf{X}^{n+1} - \mathbf{X}^n}{\delta_n} + \mathbf{R}(\mathbf{X}^{n+1}) = 0$, growing the pseudo-timestep δ_n (SER rule) as the residual falls, so early iterations are globally stable and late iterations recover Newton’s quadratic rate [?, ?]. This is an alternative to the λ -ladder for getting a hard cell to converge from a poor guess, and it degrades gracefully near the fold (it slows rather than diverges), which could replace the stall/deadline aborts.

4.5 2-D map-traversal strategies (the biggest lever)

Today the map is a bundle of independent 1-D power sweeps (§??); the frequency axis carries *no* warm information. Alternatives that use the second dimension:

- **Nearest-neighbour seeding on the grid.** Seed each new cell from the *best already-converged neighbour* in *either* axis (power or frequency), not only the cell below it. A converged cell in the adjacent frequency column is often a far better guess than a cold linear-phaser seed at the bottom of a new column.
- **Spanning-tree / flood-fill traversal.** Treat converged cells as a growing front and always solve the frontier cell whose neighbour guess is cheapest (a Dijkstra/Prim-like order over the grid), rather than a fixed column-major order. This maximizes the number of cells reached by a single warm Newton solve.
- **Boustrophedon (serpentine) ordering.** Traverse columns in alternating power direction so the end of one column is adjacent (in power) to the start of the next, enabling a warm hand-off *between* columns instead of a cold restart.
- **Two-parameter / manifold continuation.** Fold detection can be promoted to *fold-following*: continue the fold curve itself in the (f_p, P) plane as a codimension-1 manifold, mapping the exact boundary of the solvable region in one sweep instead of re-detecting it per column [?, ?].

These change only the *order* of solves and the *source* of each initial guess; the per-cell physics and corrector are untouched, so they compose with everything in §??.

4.6 Deflated continuation (find multiple branches)

If both the lower and upper (post-fold) HB branches — or genuinely distinct coexisting steady states — are of interest, *deflated continuation* systematically discovers them. After finding one solution, the residual is multiplied by a deflation operator that makes Newton *repelled* from the known root, so re-solving from the same guess converges to a *different* solution; sweeping the parameter then traces disconnected branches without a priori tangent information [?]. This is the tool of choice when a map cell may have more than one physical steady state.

4.7 Anderson / vector-extrapolation acceleration

Orthogonal to the above: Anderson acceleration mixes several previous residuals to accelerate the fixed-point/corrector iteration, often cutting iteration counts on mildly nonlinear cells for negligible cost and memory [?]. It can wrap the existing Newton–Krylov corrector rather than replace it.

4.8 Domain context: HB continuation is standard practice

Arclength/pseudo-arclength continuation wrapped around a harmonic-balance residual is the established way to compute nonlinear frequency- and amplitude-response curves with turning points, both in structural dynamics [?] and in nonlinear microwave/RF circuit simulators; JTWPA-specific modelling increasingly leans on commercial and custom HB solvers for exactly the pump-depletion / multi-harmonic regime this map probes [?, ?]. The methods above are therefore well-trodden rather than exotic.

5 Summary comparison

Method	Scope	Passes the fold?	Relation to current code
Fixed / adaptive λ continuation	intra-cell	No (stalls at fold)	<i>in use</i>
Secant predictor	intra/inter-cell	No	<i>in use</i>
Warm-start single Newton	inter-cell	No	<i>in use</i>
Tangent (Euler) predictor	intra/inter	No, but robust near it	drop-in predictor upgrade
Pseudo-arclength	intra-cell	Yes	augments residual with Eq. ??
Affine-invariant step control	intra-cell	sharper approach	replaces $\times 1.5 / \times 0.5$
Pseudo-transient (Ψ tc)	per-cell solve	graceful slow-down	alt. to λ -ladder
Neighbour / tree traversal	inter-cell (2-D)	n/a (ordering)	uses frequency axis for warm starts
Fold-following	map-level	maps fold curve	replaces patience heuristic
Deflated continuation	per-cell	finds upper branch	discovers extra solutions
Anderson acceleration	corrector	n/a	wraps Newton–Krylov

References

- [1] H. B. Keller, *Numerical solution of bifurcation and nonlinear eigenvalue problems*, in *Applications of Bifurcation Theory*, Academic Press, 1977, pp. 359–384. (Origin of pseudo-arclength continuation.)
- [2] E. L. Allgower and K. Georg, *Introduction to Numerical Continuation Methods*, Classics in Applied Mathematics 45, SIAM, 2003. <https://epubs.siam.org/doi/book/10.1137/1.9780898719154>
- [3] P. Deuffhard, *Newton Methods for Nonlinear Problems: Affine Invariance and Adaptive Algorithms*, Springer Series in Computational Mathematics 35, 2004.
- [4] C. T. Kelley and D. E. Keyes, *Convergence analysis of pseudo-transient continuation*, SIAM J. Numer. Anal. **35**(2):508–523, 1998.
- [5] C. T. Kelley, *Solving Nonlinear Equations with Newton’s Method*, SIAM, 2003.
- [6] P. E. Farrell, Á. Birkisson, and S. W. Funke, *Deflation techniques for finding distinct solutions of nonlinear partial differential equations*, SIAM J. Sci. Comput. **37**(4):A2026–A2045, 2015. <https://epubs.siam.org/doi/10.1137/140984798>
- [7] H. F. Walker and P. Ni, *Anderson acceleration for fixed-point iterations*, SIAM J. Numer. Anal. **49**(4):1715–1735, 2011.
- [8] A. Dhooge, W. Govaerts, and Yu. A. Kuznetsov, *MATCONT: A MATLAB package for numerical bifurcation analysis of ODEs*, ACM Trans. Math. Software **29**(2):141–164, 2003.
- [9] R. Veltz et al., *BifurcationKit.jl — Predictors / correctors*, documentation. <https://bifurcationkit.github.io/BifurcationKitDocs.jl/dev/Predictors/>
- [10] P. Sundararajan and S. T. Noah, *Harmonic balance method with arc-length continuation in rotor/stator contact problems*, J. Sound and Vibration / ASME (see review in *Experimental continuation in nonlinear dynamics*, arXiv:2408.00138). <https://arxiv.org/abs/2408.00138>

- [11] K. Peng et al., *Simulating the behaviour of travelling-wave superconducting parametric amplifiers using a commercial circuit simulator*, Supercond. Sci. Technol. **35**, 2022. <https://iopscience.iop.org/article/10.1088/1361-6668/ac850b>
- [12] *Modeling Josephson traveling-wave parametric amplifiers with electromagnetic and circuit co-simulation*, arXiv:2509.07807. <https://arxiv.org/abs/2509.07807>